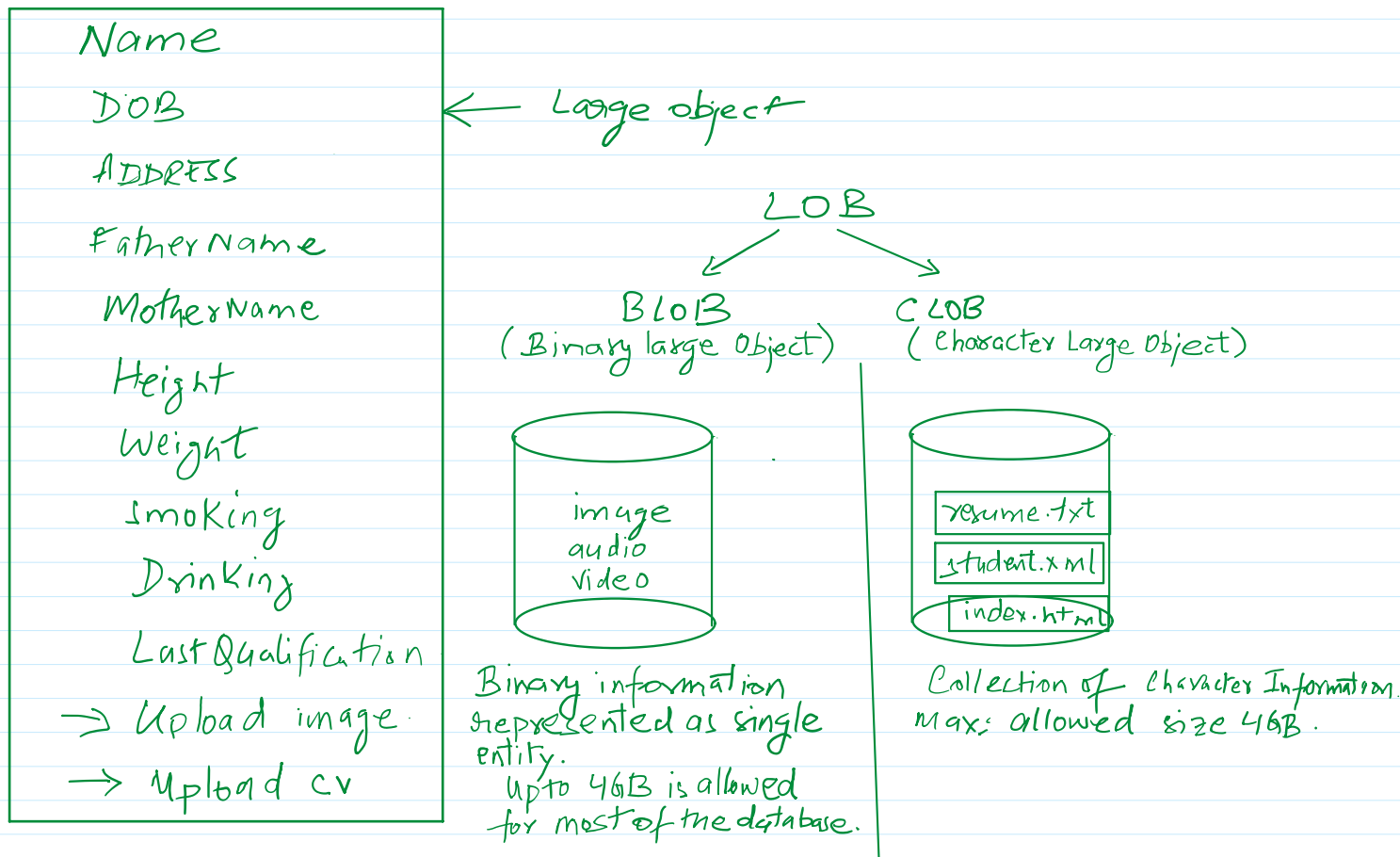


Large Objects LOB



LOBs, BLOB and CLOB Using JDBC

LOB stands for **Large Objects**. In JDBC, LOB data types are used when a database column needs to store very large amount of data such as:

- Images.
- Audio files.
- Video files.
- PDF documents.
- Word documents.
- Large text documents.
- XML/JASON/textual content.

The two most import most commonly used LOB types are:

- **BLOB - Binary large Object**
- **CLOB - Character large Object**

Oracle and MySQL both support LOB storage, although SQL data types and JDBC behavior differ somewhat.

What is LOB?

LOB (Large OBJECT) is a database data type designed to store data that is much larger than

ordinary varchar, char or binary columns.

For example, suppose we create a student table:

```
CREATE TABLE student(  
    student_id NUMBER,  
    name VARCHAR2(50),  
    photo BLOB,  
    biography CLOB  
);
```

Here:

```
student_id -> Ordinary numeric data.  
name -> Ordinary text.  
photo -> Large binary data.  
biography -> Large character data.
```

Why do we need LOBs?

Consider storing a 10 MB photograph.

VARCHAR2 would be inappropriate because the data is binary.
Instead: BLOB is designed specifically for binary data.

Similarly, if we need to store a very large text document: CLOB is appropriate.

BLOB: Is a collection of binary large data stored in a single entity in the database. Block type objects can be images, video files, audio files, etc. Blocked data type can store maximum of 4GB BINARY data.

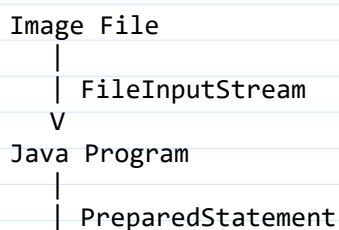
CLOB: Is a collection of character data stored in a single entity in the database. Clob can store large text documents (may be plain text or XML or HTML. Etc) CLOB type can store maximum of 4GB data. Example, history.txt

BLOB vs CLOB

Feature	BLOB	CLOB
Full form	Binary Large Object.	Character large object.
Data	Binary	Character/Text
Typical use	Images, audio files and video files.	Books, articles, XML, large text
JDBC class	java.sql.Blob	java.sql.Clob
Input Stream	InputStream	Reader
Output stream	OutputStream	Writer
Java method	setBlob()	setClob()
File type	Binary	Text

BLOB Architecture

Suppose we want to store an image.



V
JDBC Driver
|
V
Database
|
V
BLOB column

For retrieving:

BLOB Column
|
V
JDBC
|
V
Blob/InputStream
|
V
FileOutputStream
|
V
Image File

CLOB Architecture

Suppose we want to store an image.

Text File
|
| FileReader
V
Java Program
|
| PreparedStatement
V
JDBC Driver
|
V
Database
|
V
CLOB column

For retrieving:

CLOB Column
|
V
JDBC
|
V
Clob/Reader
|
V
FileWriter
|
V

Text File

```
package MyJDBCOne;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.io.Reader;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.imageio.ImageIO;
import javax.swing.ImageIcon;
import javax.swing.JFrame;
import javax.swing.JLabel;
import java.awt.image.BufferedImage;
public class ReadBLOBAnndCLOBObject {
    public static void main(String[] args) {
        // Connection con = ConnectionObject.getConnection();
        Connection con = ConnectionObject.getConnection();
        String sqlQuery="Select * FROM student_document";
        try {
            PreparedStatement ps = con.prepareStatement(sqlQuery);
            //ps.setInt(1, 101);
            ResultSet rs = ps.executeQuery();
            if (!rs.next()) {
                System.out.println("No student document found.");
                return;
            }
            int rollNo = rs.getInt("student_id");
            String nm = rs.getString("student_name");
            InputStream ist = rs.getBinaryStream("photo");
            OutputStream os = new FileOutputStream("TextFiles/retrieved_student.jpg");
            byte[] buffer = new byte[4096];
            int bytesRead;
            while((bytesRead=ist.read(buffer))!=-1){
                os.write(buffer, 0, bytesRead);
            }
            ist.close();
            os.close();

            Reader reader = rs.getCharacterStream("biography");
            char[] charBuffer = new char[4096];
            int charsRead;
            while((charsRead = reader.read(charBuffer))!= -1){
                System.out.print(new String(charBuffer, 0, charsRead));
            }
            reader.close();
            System.out.println("Roll number: " + rollNo);
            System.out.println("Name: " + nm);

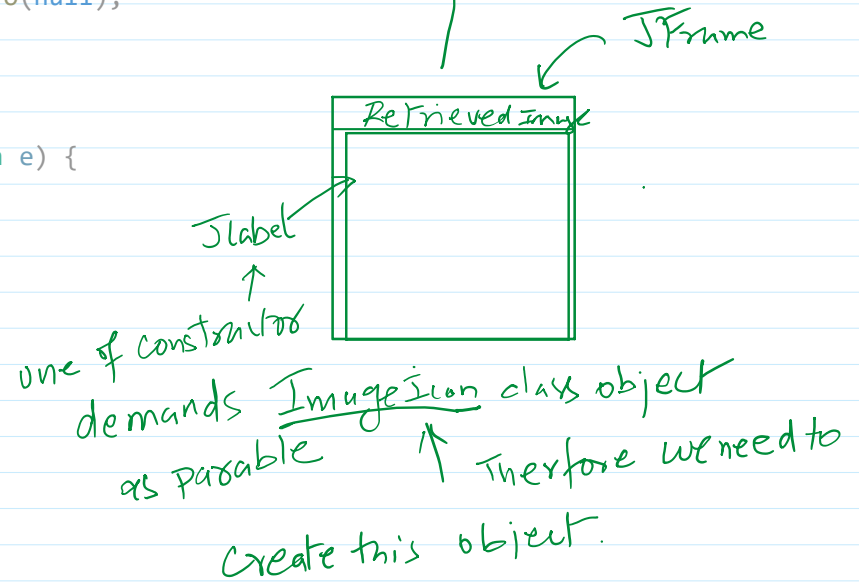
            BufferedImage img = ImageIO.read(new File("TextFiles/retrieved_student.jpg"));
            ImageIcon icon = new ImageIcon(img);
            JLabel label = new JLabel(icon);
        }
    }
}
```

✓ file object.

```

BufferedImage img = ImageIO.read(new File("TextFiles/retrieved_student.jpg"));
ImageIcon icon = new ImageIcon(img);
JLabel label = new JLabel(icon);
→ JFrame frame = new JFrame("Retrieved image");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.add(label);
frame.pack();
frame.setLocationRelativeTo(null);
frame.setVisible(true);
}
catch(SQLException e){
    e.printStackTrace();
} catch (FileNotFoundException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
}
}

```



java.awt.image. BufferedImage. ImageIcon icon = new ImageIcon(img);

→ BufferedImage Object.

BufferedImage img = ImageIO.read(fileobject)